

Anexo 20 — Benchmarks iniciales del modelo

Archivo: Scripts/benchmarks.py

Para que sirvió:

Pruebas iniciales (estabilidad, comparacion de integradores, throughput, convergencia de la calibracion) que sirvieron para decidir el enfoque de aprendizaje por refuerzo.

```
1 | """Benchmarks consolidados para asesorar Plan A vs Plan B.
2 | B1: Estabilidad con ctrl=0.
3 | B2: Sweep integrador x timestep sobre act_HipBody.
4 | B3: Throughput (mj_step/s) single-thread.
5 | B4: Convergencia actual de cada actuador con un escalon al 50% del rango.
6 | B5: Auditoria estatica MJX (texto, no requiere mjb instalado).
7 | """
8 | import sys, time, os, json
9 | import numpy as np
10 | import mujoco
11 |
12 | XML = os.path.join(os.path.dirname(__file__), "..", "Cuerpo", "DUM4.xml")
13 | XML = os.path.normpath(XML)
14 |
15 | def banner(s):
16 |     print("\n" + "="*70)
17 |     print(s)
18 |     print("="*70)
19 |
20 | def load(integrator=None, timestep=None):
21 |     """Carga modelo y opcionalmente sobrescribe integrator/timestep en runtime."""
22 |     m = mujoco.MjModel.from_xml_path(XML)
23 |     if integrator is not None:
24 |         m.opt.integrator = integrator
25 |     if timestep is not None:
26 |         m.opt.timestep = timestep
27 |     d = mujoco.MjData(m)
28 |     return m, d
29 |
30 | def get_actuator_info(m):
31 |     info = []
32 |     for a in range(m.nu):
33 |         name = mujoco.mj_id2name(m, mujoco.mjtObj.mjOBJ_ACTUATOR, a)
34 |         jid = m.actuator_trnid[a, 0]
35 |         qadr = m.jnt_qposadr[jid]
36 |         dof = m.jnt_dofadr[jid]
37 |         lo, hi = m.actuator_ctrlrange[a]
38 |         kp = m.actuator_gainprm[a, 0]
39 |         kv = -m.actuator_biasprm[a, 2]
40 |         info.append(dict(name=name, aid=a, jid=jid, qadr=qadr, dof=dof,
41 |                           ctrl_lo=lo, ctrl_hi=hi, kp=kp, kv=kv))
42 |     return info
43 |
44 | # ----- B1 -----
45 | def b1_stability():
46 |     banner("B1 - Estabilidad con ctrl=0 durante 2.0s")
47 |     m, d = load()
48 |     mujoco.mj_resetData(m, d)
49 |     nan_step = None
50 |     max_qpos = 0.0
51 |     max_qvel = 0.0
52 |     n_steps = int(2.0 / m.opt.timestep)
53 |     for i in range(n_steps):
54 |         mujoco.mj_step(m, d)
55 |         if not (np.isfinite(d.qpos).all() and np.isfinite(d.qvel).all()):
56 |             nan_step = i
57 |             break
58 |         max_qpos = max(max_qpos, np.max(np.abs(d.qpos)))
59 |         max_qvel = max(max_qvel, np.max(np.abs(d.qvel)))
60 |     # Warnings de MuJoCo
61 |     warns = []
62 |     for w in range(mujoco.mjtWarning.mjNWARNING.value):
63 |         c = d.warning[w].number
64 |         if c > 0:
65 |             wn = mujoco.mjtWarning(w).name
66 |             warns.append((wn, c))
67 |     print(f"timestep={m.opt.timestep}, integrator={mujoco.mjtIntegrator(m.opt.integrator).name}")
68 |     print(f"steps simulados: {i+1}/{n_steps} NaN en step: {nan_step}")
69 |     print(f"|qpos|_max = {max_qpos:.4f} |qvel|_max = {max_qvel:.4f}")
70 |     print(f"warnings: {warns if warns else 'ninguna'}")
71 |     return dict(ok=nan_step is None and not warns,
72 |               nan_step=nan_step, max_qpos=max_qpos, max_qvel=max_qvel, warnings=warns)
73 |
74 | # ----- B2 -----
```

```

75 | def b2_integrator_sweep():
76 |     banner("B2 - Sweep integrador x timestep sobre act_HipBody (escalon 0.4 rad)")
77 |     integrators = [
78 |         (mujoco.mjtIntegrator.mjINT_EULER.value, "Euler"),
79 |         (mujoco.mjtIntegrator.mjINT_IMPLICIT.value, "implicit"),
80 |         (mujoco.mjtIntegrator.mjINT_IMPLICITFAST.value, "implicitfast"),
81 |     ]
82 |     timesteps = [0.001, 0.002, 0.005]
83 |     target = 0.4
84 |     sim_time = 2.0
85 |     rows = []
86 |     for itg_val, itg_name in integrators:
87 |         for dt in timesteps:
88 |             m, d = load(integrator=itg_val, timestep=dt)
89 |             aid = mujoco.mj_name2id(m, mujoco.mjtObj.mjOBJ_ACTUATOR, "act_HipBody")
90 |             jid = m.actuator_trnid[aid, 0]
91 |             qadr = m.jnt_qposadr[jid]
92 |             mujoco.mj_resetData(m, d)
93 |             ts, ps = [], []
94 |             exploded = False
95 |             t_settle = None
96 |             n = int(sim_time / dt)
97 |             for i in range(n):
98 |                 d.ctrl[aid] = target
99 |                 mujoco.mj_step(m, d)
100 |                 if not np.isfinite(d.qpos[qadr]):
101 |                     exploded = True; break
102 |                 ts.append(d.time)
103 |                 ps.append(d.qpos[qadr])
104 |                 if t_settle is None and abs(ps[-1] - target) < 0.05*abs(target):
105 |                     t_settle = d.time
106 |             if exploded:
107 |                 rows.append(dict(itg=itg_name, dt=dt, status="NaN", overshoot=None,
108 |                                 t_settle=None, rms_late=None))
109 |                 continue
110 |             ps = np.array(ps); ts = np.array(ts)
111 |             overshoot = (max(ps) - target) / target * 100 if target != 0 else 0
112 |             late_mask = ts > sim_time*0.5
113 |             rms_late = float(np.sqrt(np.mean((ps[late_mask]-target)**2))) if late_mask.any() else None
114 |             rows.append(dict(itg=itg_name, dt=dt, status="ok",
115 |                             overshoot=round(overshoot,1),
116 |                             t_settle=round(t_settle,3) if t_settle else None,
117 |                             rms_late=round(rms_late,4) if rms_late else None))
118 |     # Tabla
119 |     print(f"{'integrator':14s} {'dt':>7s} {'status':>8s} {'over%':>8s} {'t_settle':>10s} {'rms_late':>10s}")
120 |     for r in rows:
121 |         over_s = "-" if r['overshoot'] is None else f"{r['overshoot']:.1f}"
122 |         sett_s = "-" if r['t_settle'] is None else f"{r['t_settle']:.3f}"
123 |         rms_s = "-" if r['rms_late'] is None else f"{r['rms_late']:.4f}"
124 |         print(f"{'itg':14s} {'dt':>7.4f} {'status':>8s} "
125 |               f"{'over_s':>8s} {'sett_s':>10s} {'rms_s':>10s}")
126 |     return rows
127 |
128 | # ----- B3 -----
129 | def b3_throughput():
130 |     banner("B3 - Throughput single-thread (mj_step/s)")
131 |     m, d = load()
132 |     mujoco.mj_resetData(m, d)
133 |     # Warm up
134 |     for _ in range(1000):
135 |         mujoco.mj_step(m, d)
136 |     N = 50000
137 |     t0 = time.perf_counter()
138 |     for _ in range(N):
139 |         mujoco.mj_step(m, d)
140 |     elapsed = time.perf_counter() - t0
141 |     sps = N/elapsed
142 |     realtime_factor = sps * m.opt.timestep # > 1 = mas rapido que real
143 |     print(f"timestep={m.opt.timestep}")
144 |     print(f"steps/s single-thread: {sps:,.0f}")
145 |     print(f"real-time factor: {realtime_factor:.1f}x")
146 |     # Proyeccion para 8 envs paralelos (estimacion lineal, overhead aparte):
147 |     print(f"con 8 envs paralelos (ideal): ~{sps*8:,.0f} steps/s -> "
148 |           f"~{sps*8*86400/1e6:.1f} Msteps/dia, "
149 |           f"~{sps*8*86400/1e6*1:.1f} M en 24h")
150 |     return dict(sps=sps, rtf=realtime_factor)
151 |
152 | # ----- B4 -----
153 | def b4_current_calibration():
154 |     banner("B4 - Convergencia con kp/kv actuales (escalon a 50% del ctrlrange, 1.5s, err<0.05)")
155 |     m, d = load()
156 |     actuators = get_actuator_info(m)
157 |     sim_time = 1.5
158 |     err_thresh = 0.05
159 |     results = []

```

```

160 | for ai in actuators:
161 |     mujoco.mj_resetData(m, d)
162 |     # Target = 50% del rango positivo o, si es bilateral, mitad superior
163 |     lo, hi = ai['ctrl_lo'], ai['ctrl_hi']
164 |     target = hi*0.5 if hi > 0 else lo*0.5
165 |     ps = []
166 |     n = int(sim_time / m.opt.timestep)
167 |     for _ in range(n):
168 |         d.ctrl[ai['aid']] = target
169 |         mujoco.mj_step(m, d)
170 |         ps.append(d.qpos[ai['qadr']])
171 |     ps = np.array(ps)
172 |     final_err = abs(ps[-1] - target)
173 |     ok = final_err < err_thresh
174 |     mse = float(np.mean((ps - target)**2))
175 |     results.append(dict(name=ai['name'], target=round(target,4),
176 |                        final=round(float(ps[-1]),4),
177 |                        err=round(final_err,4), ok=ok, kp=ai['kp'], kv=ai['kv'],
178 |                        mse=round(mse,5)))
179 |
180 | # Tabla
181 | print(f"{'actuador':25s} {'target':>8s} {'final':>8s} {'err':>8s} {'kp':>6s} {'kv':>5s} {'pass':>5s}")
182 | for r in results:
183 |     flag = "OK" if r['ok'] else "FAIL"
184 |     print(f"{r['name']:25s} {r['target']:>8.4f} {r['final']:>8.4f} {r['err']:>8.4f} "
185 |           f"{r['kp']:>6.0f} {r['kv']:>5.0f} {flag:>5s}")
186 | passed = sum(1 for r in results if r['ok'])
187 | print(f"\nPASA: {passed}/{len(results)}")
188 | return results
189 |
190 | # ----- B5 -----
191 | def b5_mjx_audit():
192 |     banner("B5 - Auditoria MJX (estatica, sobre el XML)")
193 |     # Hechos conocidos sobre MJX (a 3.x):
194 |     # - <equality joint polycoref...> -> SOPORTADO en MJX (constraint type ConstraintType.JOINT). OK.
195 |     # - <equality connect/weld> -> SOPORTADO.
196 |     # - <equality tendon> -> SOPORTADO en versiones recientes.
197 |     # - Integradores soportados en MJX: Euler, RK4, implicitfast. NO soporta 'implicit' clasico.
198 |     # - Actuators: motor, position, velocity, intvelocity, damper -> OK. muscle/cylinder limitado.
199 |     # - Sensors: la mayoria OK, algunos (rangefinder, touch grid) limitados.
200 |     # - Mesh collisions: limitadas; usar primitives o convex hulls cuando se pueda.
201 |     # - condim: todos soportados pero hay diferencias numericas con CPU.
202 |     m, _ = load()
203 |     issues, oks = [], []
204 |     # Equality
205 |     if m.neq > 0:
206 |         oks.append(f"{m.neq} equality constraints (todas polynomial JOINT) -> soportado en MJX")
207 |     # Integrator
208 |     itg_name = mujoco.mjtIntegrator(m.opt.integrator).name
209 |     if itg_name in ("mjINT_EULER", "mjINT_IMPLICITFAST"):
210 |         oks.append(f"integrador {itg_name} -> soportado en MJX")
211 |     elif itg_name == "mjINT_IMPLICIT":
212 |         issues.append(f"integrador {itg_name} -> NO soportado en MJX, cambiar a implicitfast")
213 |     else:
214 |         oks.append(f"integrador {itg_name} -> probablemente soportado, verificar docs MJX")
215 |     # Actuator types
216 |     bad_act = []
217 |     for a in range(m.nu):
218 |         gt = mujoco.mjtGain(m.actuator_gaintype[a]).name
219 |         bt = mujoco.mjtBias(m.actuator_biastype[a]).name
220 |         # position actuators usan gain=fixed, bias=affine -> soportado
221 |         if gt not in ("mjGAIN_FIXED", "mjGAIN_AFFINE", "mjGAIN_MUSCLE", "mjGAIN_USER"):
222 |             bad_act.append((a, gt, bt))
223 |     if not bad_act:
224 |         oks.append(f"{m.nu} actuators position-style (gain=fixed, bias=affine) -> soportado")
225 |     # Mesh count
226 |     nmesh = m.nmesh
227 |     oks.append(f"{nmesh} meshes STL. MJX soporta mesh collisions limitadas; "
228 |               f"si se entrena con muchos envs en GPU, considerar primitives o "
229 |               f"verificar que no haya colisiones mesh<->mesh criticas")
230 |     # Contacts auto vs explicit
231 |     ncon = m.nconmax if hasattr(m, 'nconmax') else 'auto'
232 |     oks.append(f"contactos: auto, sin <contact> explicito (puede generar muchos pares con tantos meshes)")
233 |     print("Soportado:")
234 |     for o in oks: print(f" + {o}")
235 |     print("Cambios necesarios para MJX:")
236 |     if issues:
237 |         for i in issues: print(f" ! {i}")
238 |     else:
239 |         print(" (ninguno bloqueante a nivel XML)")
240 |     return dict(oks=oks, issues=issues)
241 |
242 | # ----- main -----
243 | if __name__ == "__main__":
244 |     summary = {}
245 |     summary['B1'] = b1_stability()

```

```
245 |     summary['B2'] = b2_integrator_sweep()
246 |     summary['B3'] = b3_throughput()
247 |     summary['B4'] = b4_current_calibration()
248 |     summary['B5'] = b5_mjx_audit()
249 |     banner("RESUMEN JSON")
250 |     print(json.dumps(summary, indent=2, default=str))
251 |
```